

Lab 7:

Supporto hardware alle procedure

Esercizio 1 – strcpy (String Copy)

Scrivere una procedura RISC-V per copiare una stringa in un'altra (**strcpy**). Assumere che **dst** abbia spazio sufficiente in memoria per ricevere i byte di **src**, cioè che

strlen(dst) >= strlen(src)

Nota: **strcpy** deve utilizzare **strlen**, come in questo codice in C:

```
void strcpy(char *dst, char *src) {
    unsigned int i;
    unsigned int n;
    n = strlen(src);
    m = strlen(dst);
    for (i = 0; i < n; i++)
        dst[i] = src[i];
    for ( ; i < m; i++)
        dst[i] = '\0';
    return;
}
```

.data

src: .string "source"

dst: .string "-----"

Esercizio 1 – strcpy (String Copy)

```
.globl _start

.data
    src: .string "source"
    dst: .string "-----"

.text
_start:
    # call strcpy
    la    a0, src
    la    a1, dst
    jal   ra, strcpy

    # print the size of dst
    la    a0, dst
    jal   ra, strlen
    li    a7, 1
    ecall

    li a7, 10
    ecall
```

Main

Esercizio 1 – strcpy (String Copy)

```
*****
# strlen
# a0 - str
*****
strlen:
    add  t0, zero, zero
strlen_loop:
    add  t1, t0, a0
    lbu  t1, 0(t1)
    beq  t1, zero, strlen_end
    addi t0, t0, 1
    j    strlen_loop
strlen_end:
    addi a0, t0, 0
    ret
```

strlen

Esercizio 2 - Inverte Array

Scrivere una procedura `swap(v, x, y)` che scambi i valori di `v[x]` e `v[y]`, dove `v` è l'indirizzo di un array in memoria. Scrivere poi un'altra procedura `invert(v, s)`, che utilizzi `swap` per invertire un array in memoria.

Nota: L'indirizzo di `v` deve essere passato come parametro ad `invert` dal main, insieme a `s` (size), che rappresenta il numero di word in `v`.

- Quante istruzioni RISC-V sono necessarie per implementare la procedura?
- Quante istruzioni RISC-V verranno eseguite per completare la procedura quando l'array contiene 16 elementi?
- Quanti registri sono stati versati in memoria (*register spilling*) durante l'esecuzione?

Bonus: Realizzare un metodo `print(v, s)` che stampa `v` ad schermo

Esercizio 2 - Inverte Array

```
.globl _start
.data
    size: .word 8
    array: .word 1,5,3,7,2,6,4,8
.text
_start:
    # chiama invert
    la    a0, array
    la    a1, size
    lw    a1, 0(a1)
    jal   ra, invert

    #exit
    li    a7, 10
    ecall
```

Main

Esercizio 2 - Inverte Array

```
# Procedure swap(v, x, y)
# a0 -> address of v
# a1 -> index x
# a2 -> index y
swap:
    slli    a1, a1, 2      # calculates offset of x
    slli    a2, a2, 2      # calculates offset of y
    add     t0, a0, a1     # address of v[x]
    add     t1, a0, a2     # address of v[y]

    lw      t2, 0(t0)      # swap the values
    lw      t3, 0(t1)
    sw      t3, 0(t0)
    sw      t2, 0(t1)
    ret                                # return
```

swap



Quando il compilatore incontra una procedura foglia (come `swap`) esaurisce tutti i registri temporanei prima di utilizzare quelli di cui deve salvare il contenuto

Esercizio 3 – Fibonacci (ricorsivo)

Tradurre il seguente frammento di codice C in codice assembly RISC-V.

```
int fib(int n) {  
    if (n==0)  
        return 0;  
    else if (n==1)  
        return 1;  
    else  
        return fib(n-1) + fib(n-2);  
}
```


Esercizio 3 – Fibonacci (ricorsivo)

Main:

```
.globl _start
.data
    num: .word 5
.text
_start:
    # call fib
    la    a0, num
    lw    a0, 0(a0)
    jal   ra, fib

    li a7, 1
    ecall
    #exit
    li    a7, 10
    ecall
```